

Test Case Prioritization using Ant Colony Optimization

Yogesh Singh

University School of Information Technology

Guru Gobind Singh Indraprastha University, Kashmere Gate, Delhi-110403, India

e-mail: ys66@rediffmail.com

Arvinder Kaur

University School of Information Technology

Guru Gobind Singh Indraprastha University, Kashmere Gate, Delhi-110403, India

e-mail: arvinderkaurtakkar@yahoo.com

Bharti Suri

University School of Information Technology

Guru Gobind Singh Indraprastha University, Kashmere Gate, Delhi-110403, India

e-mail: bhartisuri@gmail.com

Abstract

Regression testing is primarily a maintenance activity that is performed frequently to ensure the validity of the modified software. In such cases, due to time and cost constraints, the entire test suite cannot be run. Thus, it becomes essential to prioritize the tests in order to cover maximum faults in minimum time. In this paper, ant colony optimization is used, which is a new way to solve time constraint prioritization problem. This paper presents the regression test prioritization technique to reorder test suites in time constraint environment along with an algorithm that implements the technique.

Keywords: Regression Testing, Prioritization, Ant Colony Optimization

1. Introduction

Regression testing is performed to ensure that modifications are correct and have not adversely affected the unchanged portions of the software. The test suites that are already available from earlier versions of software can be expensive to execute completely. Many techniques are proposed by researchers for reducing cost related to regression testing. They include regression test selection [13, 20], regression test prioritization [11, 14, 21, 22, 25], and hybrid approach[24]. In selective regression testing technique, only a subset of test cases is selected and re run. In prioritizations, test cases are ordered such that the highest priority test case is executed earlier than lower priority test cases. The priority is set according to chosen criteria to increase rate of fault detection or to achieve maximum code coverage or to cover important features earlier and so on. When execution time is short, we need to have time constraint prioritization technique. The time constrained test case prioritization problem can be reduced to NP-complete 0/1 knapsack problem [14]. NP-completeness applies to combinatorial optimization problems. Ant Colonies(AC's) is a general purpose heuristics that has been widely used in solving combinatorial optimization problem like classic travelling salesman problem, data mining, telecommunication networks, vehicle routing[2, 3,4,

6, 12,15,16,19,26] etc. Thus, time constraint test case prioritization problem can be solved using Ant Colony Optimization (ACO).

2. Ant Colony Optimization

The concept of ACO has been inspired by observation of real ants while wandering for food source. An ant while in search of a food source (target) lays a certain amount of pheromone trail (an odorous substance) along the path traversed. The laying of pheromone serves dual purpose to the ant. Firstly, it marks the path for other randomly moving ants and secondly, it helps the ant to return to its original source. Now, whenever a redundantly moving ant senses pheromone fragments it is likely that it will follow the same path and hence by mechanism of positive feedback this new ant lays more pheromone on that path. The amount of pheromone laid by the new ant will intensify the amount of already present pheromone trail on a specific path. A path which has maximum pheromone trail will be chosen by a new arbitrarily moving ant. The pheromone laid evaporates at a constant rate. This evaporation also helps in avoiding the convergence to a local optimal solution. If any ant randomly moves on the shortest path, that ant will be the first one to reach its original source also. Now in this forth and back traversal of the ant on the shortest path it will lay more pheromone trail, making that path an obvious choice for other ants after a few hit and trials and hence quickly converging to the shorter path. This behaviour of ants from its colony to food source and back to its colony is called ant colony optimization.

3. ACO approach to solve combinatorial optimization problems

Ants, on being blind and small in size are able to find the shortest route to their food source. They use antennas and food trail pheromone to be in touch with each other. ACO technique has been derived from the behavior of real ants, and their synchronization with searching solutions to local problem and

maintaining array list for maintaining previous information brought up by each ant. ACO can be used to solve combinatorial optimization problems (COP) [9]. The COP can be defined as:

If C is a set of components, $C = c_1, \dots, c_n$ and the solution of a problem is given by subset S of components then subset of feasible solution is given by $F \subseteq 2^C$. It means solution S is possible iff $S \in F$. A cost function z is defined over solution domain, $z: 2^C \rightarrow R$.

The main objective is to search minimum cost possible solution S^* , i.e. to find $S^*: S^* \in F$ and $z(S^*) < z(S)$, $\forall S \in F$. In ACO, ants make a solution for combinatorial optimization problem by going through each path and making a construction graph, $G_c(V, E)$ [9]. The graph is thoroughly connected and contains set of vertices V and edges E . The set of components C can be either associated with V or E . Ants build partial solution by going through each vertex using edges, meanwhile depositing pheromone on components (on either V or E). This act of making trail of pheromone is used as important information in traversing toward food source. Figure 1 represents how ants move from source to destination and select optimal path.

The ants are able to find shortest path on the basis of (i) the pheromone laid on ground by other ants of same colony which represents the experience of the colony and (ii) memory and heuristic information which represents useful knowledge about the particular problem the ants are solving. Therefore, probability of next path chosen by k^{th} ant is given by: [23]

$$\rho_{ij}^k(t) = \tau_{ij}^\alpha \eta_{ij}^\beta / \sum_{k \in \tau} \tau_{ij}^\alpha \eta_{ij}^\beta \quad j \in \tau$$

$$\rho_{ij}^k(t) = 0 \quad \text{otherwise}$$

where,

τ_{ij}^α is pheromone level on path (i, j) of graph,

η_{ij}^β is heuristic function and

τ is array list (memory of ant) containing all the trails that the ant has already passed and must not be chosen again.

Heuristic function η is used to search for optimal path with the help of useful information, and the array list τ stores the previous paths visited in the memory.

Updating the trail is performed when ants complete their search of food source. The updating of pheromone is achieved by using [23]:

$$\tau_{ij} \leftarrow (1 - \rho) \tau_{ij} + \rho \sum_{k=1}^g \Delta \tau_{ij}$$

Where, ρ is pheromone trail evaporation that lies in [0, 1] and $\sum_{k=1}^g \Delta \tau_{ij}$ are pheromone deposited in the trail (i, j) followed by g ants after completing a tour.

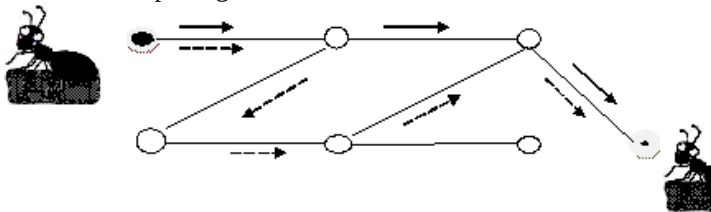


Fig 1: Ants exploring various paths. Solid arrows represents optimal path

There is another important process, i.e. food trail pheromone evaporation that is decreasing in amount of pheromone deposited on every path with time [18].

4. Time constraint Test Suite Prioritization as NP complete problem

A selective regression testing technique chooses a subset of test suite that was used to test the software before modifications were made, and then uses this subset to test the modified software [13,25]. Prioritization is the process of scheduling test cases in an order to meet some performance goal.

We define a test suite T as a tuple of test cases T_i from $i=1$ to n as $(T_1, T_2, \dots, T_n)$. The prioritization of T is denoted by T' . With Knapsack problem, the maximum time within which a prioritize test suite must execute is the maximum capacity of knapsack i.e. test cases are knapsack items, execution time of each test case is its weight and its value is percentage of fault coverage. The knapsack 0/1 algorithm outputs prioritized list that fits the required time limits. Formally, 0/1 knapsack in terms of test suite prioritization is defined as [1]:

$$\text{Maximize: } \sum_{i=1}^n c_i x_i$$

$$\text{Subject to: } \sum_{i=1}^n t_i x_i \leq t_{\max}, x_i = 0 \text{ or } 1$$

where,

c_i is fault coverage,

t_i is execution time of test case T_i

$t_{\max}$ is maximum time allowed for the execution of the prioritization.

Thus, the 0/1 knapsack problem is an NP-complete problem [22].

All NP complete problems are NP hard. Ant colony optimization is used as a method for solving hard combinatorial optimization problems (COP's) [10].

5. Test suite selection and prioritization using ACO

For a given test suite, the problem of selection and prioritization of test cases can be stated as follows:-

Given the original set 'T' (Test Suite) of 'n' test cases, find subset 'S', which consists of 'm' test cases ($m < n$, $S \subseteq T$), such that the test cases are selected and prioritized on the basis of maximum fault coverage capacity of the test cases.

Assumptions –

1. Original test suite, $T = \{t_1, t_2, \dots, t_n\}$
2. Set of all faults, $F = \{f_1, f_2, \dots, f_x\}$
3. Each test case $\{t_1, t_2, \dots, t_n\}$ in the original test suite covers some or all the faults from 'F'.
4. ' z_i ' is the time elapsed for each ant as it moves from one node (test case) to other. TC, is the total time constraint put on the selection and prioritization problem i.e., total time constraint for evaluating complete path of an ant. It is set to constant MAX.

5. Number of artificial ants to search through the test case space is n (number of test cases).
6. For each ant 'j', a list that contains the selected test cases (and thus faults covered on the current path) is represented as $S_j = \{s_1, s_2, \dots, s_m\}$.
7. w_i is the weight of each edge 'i', which is assumed to be the amount of pheromone deposited on the edge.
8. Assume pheromone deposition rate to be +1 or 100% for each ant that has crossed the edge on a best path.
9. Assume pheromone evaporation rate to be $k\%$ of w_i (current weight of i^{th} edge) to be reduced for each edge after each iteration of the loop. ($k = 10\%$ for our algorithm [10])

6. Problem Representation

The problem can be represented in the form of an undirected graph $G(V, E)$ where V is the set of vertices and E is the set of edges in the graph. $V \equiv T$ i.e. test cases are represented by the vertices in the graph. ' w_i ' is the weight of ' i^{th} ' edge in the graph. It represents the pheromone trail associated with the edge $e_i \in E$, which reflects the amount of fault coverage ' f_i ' on the chosen path within time constraint, 'TC'. Initially it is set to zero for all edges.

7. Proposed Algorithm

1. Initialization:
Set $w_i = 0$
Set $TC = MAX$
Set $Z_k = 0$ for all $k=1$ to n
Generate 'n' number of artificial ants $\{a_1, a_2, \dots, a_n\}$
 $S_1, \dots, S_n = \emptyset$
2. Do
For $k = 1$ to n
 $S_k = S_k + \{t_k\}$ //The initial test case t_k for ant a_k is the
 $tmpT = t_k$ //starting vertex for ant a_k on the graph.
 Do
 $t = \text{Call select_test_case}(a_k, tmpT)$
 Update $S_k = S_k + \{t\}$
 $Z_k = Z_k + \text{execution time for } t$
 $tmpT = t$
 While (total faults covered);
EndFor
 $minTime = \min\{Z_k\}$ where $k=1$ to n
 $maxTime = \max\{Z_k\}$ where $k=1$ to n
 $currTime = currTime + maxTime$
 $Z_k = 0 \forall k = 1$ to n
 Update pheromone at all the edges for the best path corresponding to $minTime$
 Evaporate $k\%$ ($=10\%$) of pheromone for each edge where $w_i \geq 0$
 $S_k = \emptyset, \forall k = 1$ to n
 While ($TC \geq currTime$);
Select_test_case($a_k, tmpT$)
{
 If there is no pheromone deposited on the edge starting from node $tmpT$
 and ending at node $\in S_k$ and is starting from $tmpT$
 Then choose random edge.
 Else if (number of edges starting from node $tmpT$ and ending at node $\in S_k$
 having max pheromone is one)
 Then return that edge.
Else

Return an edge randomly selected among edges starting from $tmpT$ ending at node $\in S_k$ having maximum pheromone.
}

8. Example

Consider a test suite with 8 test cases in it, covering a total of 10 faults. Test prioritization schemes typically create a single reordering of the test suite that can be executed after many subsequent changes to the program under test. Reordering of test suite can be more effective at finding faults if testing must be terminated earlier [8]. In this section we present the execution of our algorithm in time based prioritization.

The regression test suite T contains eight test cases with the initial ordering $\{T1, T2, T3, T4, T5, T6, T7, T8\}$ as described in Table 1. This example assumes a priori knowledge of the faults detected by T in the program P . As shown in Table 2, test case $T1$ can find four faults, $\{f2, f4, f7, f9\}$ in seven minutes, $T2$, finds two fault, $\{f1, f3\}$ in four minutes, and $T3$ finds four faults, $\{f1, f5, f7, f8\}$ in five minutes. Test cases $T4$ and $T5$ find three faults in four minutes, $\{f2, f4, f9\}$ and $\{f3, f6, f10\}$. Test cases $T6$ and $T8$ find two faults in five and two minutes, $\{f1, f7\}$ and $\{f2, f10\}$. Test case $T7$ finds three faults in four minutes, $\{f3, f6, f8\}$ respectively. Each test case covers the faults as shown in the table below:

Test case/ faults	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10
T1		X		X			X		X	
T2	X		X							
T3	X				X		X	X		
T4		X		X					X	
T5			X			X				X
T6	X						X			
T7			X			X		X		
T8		X								X

Table 1: Sample Test cases, Faults identified and its Execution time.

TEST CASE	NO.OF FAULTS COVERED	EXECUTION TIME(UNIT)
T1	4	7
T2	2	4
T3	4	5
T4	3	4
T5	3	4
T6	4	5
T7	3	4
T8	2	2

Table 2: Sample Test cases, Faults identified and its Execution time.

Total time limit assumed to be 85 time units. This is the stopping criteria for execution of algorithm. The paths traversed by various

ants are shown in table 3 to table 6. Table 3 displays status of paths explored after 1st iteration.

A N T S	OBSERVATIONS FOR 1 ST ITERATION						TOTAL TIME (UNITS)
A1	P	T1	T3	T6	T7	T8	23
	T	7	5	5	4	2	
	FC	2,4,7,9	1,5,7,8	1,7	3,6,8	2,10	
A2	P	T2	T5	T4	T7	T3	21
	T	4	4	4	4	5	
	FC	1,3	3,6,10	2,4,9	3,6,8	1,5,7,8	
A3	P	T3	T7	T5	T8	T4	19
	T	5	4	4	2	4	
	FC	1,5,7,8	3,6,8	3,6,10	2,10	2,4,9	
A4	P	T4	T6	T3	T5		18
	T	4	5	5	4		
	FC	2,4,9	1,7	1,5,7,8	3,6,10		
A5	P	T5	T7	T1	T3		20
	T	4	4	7	5		
	FC	3,6,10	3,6,8	2,4,7,9	1,5,7,8		
A6	P	T6	T4	T1	T5	T3	25
	T	5	4	7	4	5	
	FC	1,7	2,4,9	2,4,7,9	3,6,10	1,5,7,8	
A7	P	T7	T8	T1	T3		18
	T	4	2	7	5		
	FC	3,6,8	2,10	2,4,7,9	1,5,7,8		
A8	P	T8	T3	T5	T7	T4	19
	T	2	5	4	4	4	
	FC	2,10	1,5,7,8	3,6,10	3,6,8	2,4,9	

P: Path, T: Execution time, FC: Faults Covered

Table 3: Execution of algorithm (1st Iteration) with table 1 as input.

In the current iteration, all the 8 artificial ants select random paths and stop when a total fault coverage criterion is met. Here, we observe that the ants A4 and A7 discover the best path (as they take minimum total time). Hence, pheromone has been updated (+1 added) on those edges as shown in figure. Then 10% of pheromone is evaporated from each edge. The best path discovered after 1st iteration is shown in figure 2.

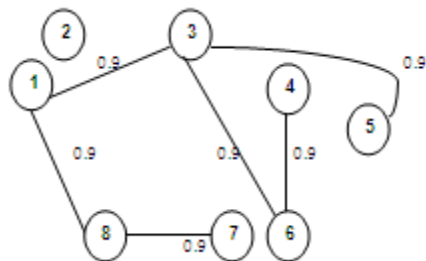


Figure 2: Graph representing pheromone deposited after 1st iteration.

The paths explored and the execution time for each path after 2nd iteration is shown in Table 4. The execution time after 2nd iteration is 49(25+24) time units. The corresponding graph for pheromone deposition is shown in Figure 3. For edge 3-5(which is also lying on the best path of current iteration), new weight (pheromone level) is calculated as: $W = 0.9 + 1 - 10\% (0.9 + 1) = 1.71$.

P: Path, T: Execution time, FC: Faults Covered

A N T S	OBSERVATIONS FOR 2 ND ITERATION						TOTAL TIME (UNITS)
A1	P	T1	T8	T6	T3	T5	23
	T	7	2	5	5	4	
	FC	2,4,7,9	2,10	1,7	1,5,7,8	3,6,10	
A2	P	T2	T3	T5	T4		17
	T	4	5	4	4		
	FC	1,3	1,5,7,8	3,6,10	2,4,9		
A3	P	T3	T1	T8	T7		18
	T	5	7	2	4		
	FC	1,5,7,8	2,4,7,9	2,10	3,6,8		
A4	P	T4	T6	T3	T5		18
	T	4	5	5	4		
	FC	2,4,9	1,7	1,5,7,8	3,6,10		
A5	P	T5	T3	T6	T4		18
	T	4	5	5	4		
	FC	3,6,10	1,5,7,8	1,7	2,4,9		
A6	P	T6	T4	T3	T5		18
	T	5	4	5	4		
	FC	1,7	2,4,9	1,5,7,8	3,6,10		
A7	P	T7	T8	T1	T3		24
	T	4	2	7	5		
	FC	3,6,8	2,10	2,4,7,9	1,5,7,8		
A8	P	T8	T7	T6	T3	T1	23
	T	2	4	5	5	7	
	FC	2,10	3,6,8	1,7	1,5,7,8	2,4,7,9	

Table 4: Execution of algorithm (2nd iteration)

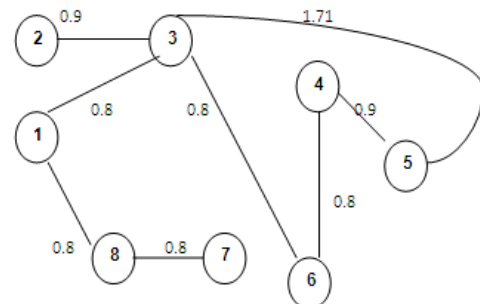


Figure 3: Graph representing pheromone deposited after 2nd iteration

The algorithm stops executing after 4th iteration as time constraint limit exceeds TC (85 time units). Table 5 and 6 show the paths discovered after 3rd and 4th iteration. The corresponding graphs are represented in figure 4 and 5. The best path discovered after 4th iteration is {T3, T5, T4} and the minimum execution time has been calculated to be 16 time units. This path is comparable to the optimal path that can be obtained for this example. In future we expect to achieve the same results for real examples due to the robust nature of ACO technique

ANT S	OBSERVATIONS FOR 3 rd ITERATION					TOTAL TIME (UNITS)
A1	P	T1	T3	T5		16
	T	7	5	4		
	FC	2,4,7,9	1,5,7,8	3,6,10		
A2	P	T2	T3	T5	T4	17
	T	4	5	4	4	
	FC	1,3	1,5,7,8	3,6,10	2,4,9	
A3	P	T3	T5	T4		13
	T	5	4	4		
	FC	1,5,7,8	3,6,10	2,4,9		
A4	P	T4	T5	T3		13
	T	4	4	5		
	FC	2,4,9	3,6,10	1,5,7,8		
A5	P	T5	T3	T2	T1	20
	T	4	5	4	7	
	FC	3,6,10	1,5,7,8	1,3	2,4,7,9	
A6	P	T6	T4	T5	T3	18
	T	5	4	4	5	
	FC	1,7	2,4,9	3,6,10	1,5,7,8	
A7	P	T7	T8	T1	T3	18
	T	4	2	7	5	
	FC	3,6,8	2,10	2,4,7,9	1,5,7,8	
A8	P	T8	T1	T3	T5	18
	FC	2,10	2,4,7,9	1,5,7,8	3,6,10	

P: Path, T: Execution time, FC: Faults Covered

Table 5: Execution of algorithm (3rd iteration)

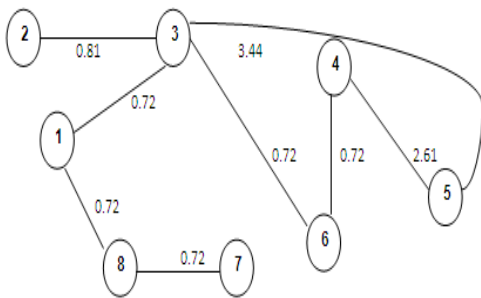


Figure 4: Graph representing pheromone deposited after 3rd iteration

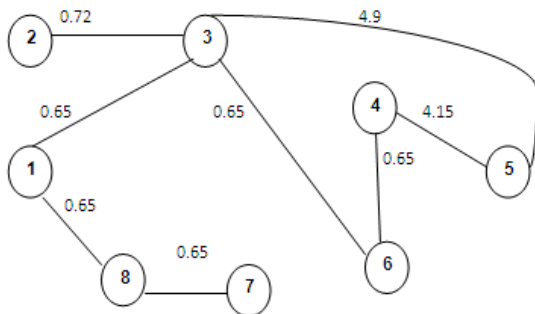


Figure 5: Graph representing pheromone deposited after 3rd iteration

ANT S	OBSERVATIONS FOR 4 th ITERATION						TOTAL TIME (UNITS)
A1	P	T1	T3	T5			18
	T	7	5	4			
	FC	2,4,7,9	1,5,7,8	3,6,10			
A2	P	T2	T3	T5	T4		20
	T	4	5	4	4		
	FC	1,3	1,5,7,8	3,6,10	2,4,9		
A3	P	T3	T5	T4			16
	T	5	4	4			
	FC	1,5,7,8	3,6,10	2,4,9			
A4	P	T4	T5	T3			20
	T	4	4	5			
	FC	2,4,9	3,6,10	1,5,7,8			
A5	P	T5	T3	T2	T4		16
	T	4	5	4	4		
	FC	3,6,10	1,5,7,8	1,3	2,4,9		
A6	P	T6	T3	T5	T4		31
	T	5	5	4	4		
	FC	1,7	1,5,7,8	3,6,10	2,4,9		
A7	P	T7	T8	T1	T3		20
	T	4	2	7	5		
	FC	3,6,8	2,10	2,4,7,9	1,5,7,8		
A8	P	T8	T7	T4	T5	T3	35
	T	2	4	4	4	5	
	FC	2,10	3,6,8	2,4,9	3,6,10	1,5,7,8	

P: Path, T: Execution time, FC: Faults Covered

Table 6: Execution of algorithm (4th iteration)

Comparison:

The example mentioned in above section is compared with respect to the following ordering: No order, Random order, Reverse order, optimal order of the test cases. The orderings with respect to these approaches are listed in Table 7. These approaches are compared by calculating Average Percentage of Faults Detected (APFD) [11,20, 21] using the same example. The results are shown in fig. 6 to fig. 10. It shows that the prioritization achieved using ACO attain the results that are comparable to that obtain with optimum ordering. The technique is superior to other approaches also as is clearly shown with the percentage of fault coverage achieved.

No Order	Random Order	Reverse Order	Optimum Order	ACO Order
T1	T5	T8	T1	T3
T2	T7	T7	T3	T5
T3	T1	T6	T5	T4
T4	T3	T5	T4	T1
T5	T6	T4	T6	T7
T6	T2	T3	T7	T8
T7	T4	T2	T8	T6
T8	T8	T1	T2	T2

Table 7: Order of test cases for various prioritization approaches.

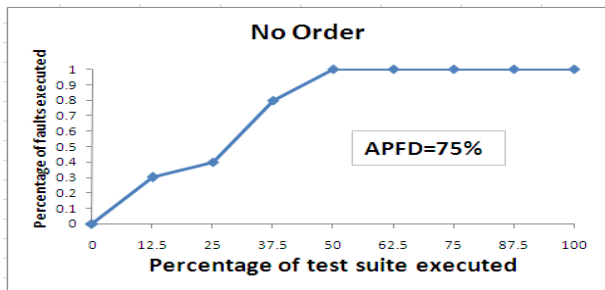


Fig. 6 APFD for original ordering of test cases

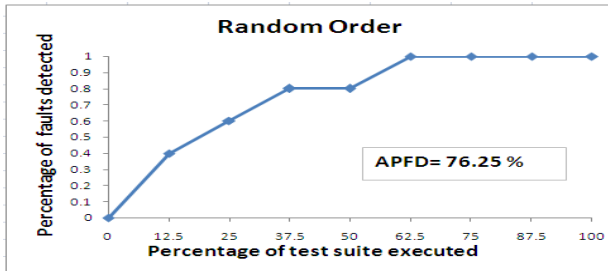


Fig. 7 APFD for random ordering of test cases

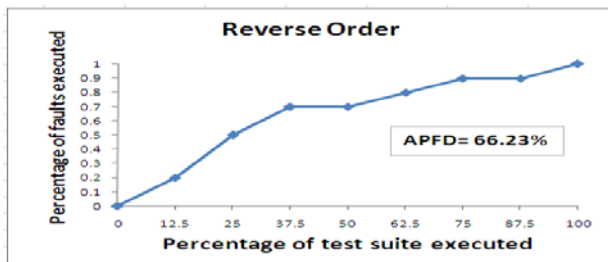


Fig.8 APFD for reverse ordering of test cases

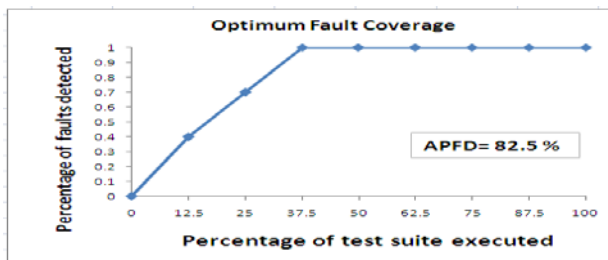


Fig9 APFD for Optimal Fault Coverage ordering of test cases

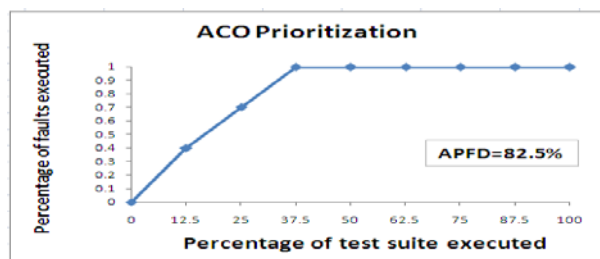


Fig.10 APFD for ordering using ACO Prioritization of test cases

9. Application of the Proposed Approach

Software practitioners may use the algorithm developed to reduce the time & effort required for prioritization of test cases. This algorithm may lead to greater time & cost savings when applied to larger & complex test suites, as compared to the smaller ones. Using ACO approach, software practitioners can effectively select & prioritize test cases from a test suite, with minimum execution time. Hence, the proposed algorithm may prove to be useful in real-life situations.

10. Conclusion

We have proposed selection & prioritization approach based on ACO to find the solution that is nearest to optimal solution. Our preliminary results on the example considered in the paper, gave encouraging results. In comparison of ACO with other techniques for the example by calculating APFD (Average Percentage of Faults Detected) for each technique, we have found that prioritization achieved using ACO results in ordering that is comparable to optimum ordering. ACO is strong & robust as it involves positive feedback and parallel computations and hence, it can lead to better solutions in optimum time. Issue of future research includes automation of the technique and applying it on large and complex software. We also aim to compare it to genetic based prioritization.

References

1. Alspaugh, S., Wolcott, K.R, Belanich, M., Kapfhamer, G.M. and Lou Soffa, M. (2007): Efficient Time Aware Prioritization with Knapsack Solvers, ACM New York, USA, pp.13-18.
2. Ayari, K., Bouktif, S. and Antoniol, G. (2007): Automatic Mutation Test Input Data Generation via Ant Colony. pp:1074
3. Caro, G. Di and Dorigo, M., (1998): AntNet: Distributed stigmergetic control for communications networks. Journal of Artificial Intelligence Research, 9, pp. 317-365.
4. Caro, G. Di and Dorigo, M., (1998): Antnet: Distributed stigmergetic control for communications networks. Journal of Artificial Intelligence Research, 9, pp. 317-367.
5. Coloni, A., Dorigo, M., and Maniezzo, V., (1991): Distributed optimization by ant colonies. In Proceedings of ECAL'91, European Conference on Artificial Life, Elsevier Publishing, Amsterdam.
6. Dorigo, M., Maniezzo, V. and Coloni, A. (1996): Ant System: Optimization by a colony of cooperating agents. IEEE Transactions on Systems, Man and Cybernetics, B(26), pp.29-41.

7. Dorigo, M., (1992): Optimization, learning and natural algorithms, Ph.D. Thesis, Politecnico di Milano, Milano.
8. Dorigo, M., Maniezzo, V. and Coloni, A., (1991): The Ant System: An Autocatalytic Optimizing Process, Technical Report TR91-016, Politecnico di Milano.
9. Dorigo, M. <http://iridia.ulb.ac.be/mdorigo/ACO/ACO.html>
10. Dorigo, M. and Socha, K. (2007): An Introduction to Ant Colony Optimization, CRC Press.
11. Elbaum S., Rothermel G., Kanduri S., Malishevsky A.G., (2004): Selecting a cost-effective test case prioritization technique. *Software Quality Journal* 12(3), pp.185-210.
12. Gomez, O. and Baren, B. (2005): Omicron ACO. A New Ant Colony Optimization Algorithm. *clei electronic journal*, 8(1), paper 5.
13. Graves, T. L., Harrold, M. J., Kim, M.J., Porter, A. and Rothermel, G. (2001): An empirical study of regression test selection techniques. *ACM Trans. on Softw. Eng. and Meth.* 10(2).
14. Krishnamoorthi, R., Sahaaya, S.A. and Mary, A. (2009): Regression Test Suite Prioritization using Genetic Algorithms. *International Journal of Hybrid Information Technology*, 2(3), pp. 35.
15. Li, H. and Peng Lam, C. (2005): Software Test Data Generation Using Ant Colony Optimization. pp: 1.
16. Li, L., Ju, S. and Zhang, Y. (2008): Improved Ant Colony Optimization for the Travelling Salesman Problem. *International Conference on Intelligent Computation Technology and Automation*, pp. 76.
17. Maniezzo, V., (1999): Exact and approximate nondeterministic tree-search procedures for the quadratic assignment problem, *INFORMS Journal of Computing* 11(4), pp.358—369.
18. Maniezzo, V., Gambardella, L.M. and De Luigi, F., (2004): Ant Colony Optimization. *New Optimization Techniques in Engineering*, by Onwubolu, G. C., and B. V. Babu, Springer-Verlag Berlin Heidelberg, pp. 101-117.
19. Parpinelli, R.S., Lopes, H.S. and Freitas, A.A., (2002): Data mining with an ant colony optimization algorithm. *IEEE Transactions on Evolutionary Computation*, 6, pp. 321 – 332.
20. Rothermel G., Harrold M.J., Dedhia J., (2000): Regression test selection for C++ programs, *Software Testing, Verification and Reliability* 10(2). pp, 77-109.
21. Rothermel Gregg, Untch R.H., Chu Chengyun, Harrold Mary Jean, (Sept 1999): Test Case Prioritization: An Empirical Study, in: *Proceedings of the International Conference on Software Maintenance*, Oxford, UK, 179-188.
22. Rothermel, G., Untch, R. J. and C. Chu, (2001): Prioritizing test cases for regression testing. *IEEE Transaction on Software. Eng.*, 27(10), pp. 929-948.
23. Silva, C.A. and Runkler, T.A., (2009): Ant Colony Optimization for dynamic Traveling Salesman Problems. Siemens AG, Corporate Technology Information and Communications, CT IC 481730 Munich – Germany.
24. Singh Y., Kaur, A. and Suri B. (2006): A New Technique for Version – Specific Test Case Selection and Prioritization for Regression Testing. *Journal of Computer Society of India*, 36(4), pp. 23-32.
25. Walcott, K.R., Lou Soffa, M., Kapfhammer, G.M. and Robert S. Roos, (2006): Time Aware Test Suite Prioritization. pp, 1-2.
26. Zhao, P., Zhao, P. and Zhang, X. (2006): New Ant Colony Optimization for the Knapsack Problem.